========== MODULE 1: SUPPORT VECTOR MACHINES (SVM) - 5 NUMERICAL QUESTIONS

Question 1.1 (Hard-Margin SVM Weights and Bias Optimization) You are given a 2D dataset with four training samples:

- x1 = [1, 2]^T, y1 = +1
- x2 = [2, 1]^T, y2 = +1
- x3 = [3, 4]^T, y3 = -1
- x4 = [4, 3]^T, y4 = -1 Assume that the optimal separating hyperplane is determined exclusively by the support vectors x1, x2, and x3. Find the exact numerical values for the weight vector w = [w1, w2]^T and the bias b that satisfy the margin equations y_n(w^T x_n + b) = 1.

Solution: For support vectors, the margin constraint holds with strict equality: (1) For x1: 1 * (w1(1) + w2(2) + b) = 1 => w1 + 2w2 + b = 1 (2) For x2: 1 * (w1(2) + w2(1) + b) = 1 => 2w1 + w2 + b = 1 (3) For x3: -1 * (w1(3) + w2(4) + b) = 1 => 3w1 + 4w2 + b = -1

Subtract (1) from (2): (2w1 + w2 + b) - (w1 + 2w2 + b) = 1 - 1 w1 - w2 = 0 => w1 = w2

Substitute w1 = w2 into (1) and (3): From (1): w1 + 2w1 + b = 1 => 3w1 + b = 1 => b = 1 - 3w1 From (3): 3w1 + 4w1 + b = -1 => 7w1 + b = -1

Substitute b = 1 - 3w1 into the updated equation (3): 7w1 + (1 - 3w1) = -1 4w1 + 1 = -1 => 4w1 = -2 => w1 = -0.5 Since w1 = w2, then w2 = -0.5.

Now find b: b = 1 - 3(-0.5) = 1 + 1.5 = 2.5 The optimal parameters are w = [-0.5, -0.5]^T and b = 2.5.

Question 1.2 (Soft-Margin SVM Slack Variables and Hinge Loss Evaluation) Consider a soft-margin SVM classifier with w = [1.0, -2.0]^T and b = 0.5. Calculate the exact numerical values of the Hinge Loss = max(0, 1 - y_n(w^T x_n + b)) and the slack variables ξ_n for the following points:

- xA = [1, 1]^T with true label yA = +1
- xB = [2, 0]^T with true label yB = -1
- xC = [0, -1]^T with true label yC = -1

Solution: The decision score formula is f(x) = 1.0*x1* - *2.0*x2 + 0.5.

- For xA: f(xA) = 1(1) - 2(1) + 0.5 = -0.5 yA * f(xA) = (+1) * (-0.5) = -0.5 Loss_A = max(0, 1 - (-0.5)) = max(0, 1.5) = 1.5. Thus, ξ_A = 1.5.
- For xB: f(xB) = 1(2) - 2(0) + 0.5 = 2.5 yB * f(xB) = (-1) * (2.5) = -2.5 Loss_B = max(0, 1 - (-2.5)) = max(0, 3.5) = 3.5. Thus, ξ_B = 3.5.
- For xC: f(xC) = 1(0) - 2(-1) + 0.5 = 2.5 yC * f(xC) = (-1) * (2.5) = -2.5 Loss_C = max(0, 1 - (-2.5)) = max(0, 3.5) = 3.5. Thus, ξ_C = 3.5.

Question 1.3 (Functional vs Geometric Margin Calculation) A linear separating hyperplane is parameterized by w = [3, 4]^T and b = -5. For a training point x = [2, 1]^T with label y = +1, compute:

1. The functional margin γ_f = y(w^T x + b).
2. The geometric margin γ_g = y(w^T x + b) / ||w||.

Solution:

1. Functional Margin: w^T x + b = (3 * 2) + (4 * 1) - 5 = 6 + 4 - 5 = 5 γ_f = y * (5) = (+1) * 5 = 5
2. Geometric Margin: Calculate the Euclidean norm of w: ||w|| = sqrt(3^2 + 4^2) = sqrt(9 + 16) = sqrt(25) = 5 γ_g = γ_f / ||w|| = 5 / 5 = 1.0

Question 1.4 (Kernel SVM Feature Mapping Inner Product) Let φ(x) be a quadratic feature mapping for 2D vectors x = [x1, x2]^T defined as φ(x) = [x1^2, sqrt(2)x1x2, x2^2]^T. Given two data vectors a = [2, 3]^T and b = [1, 4]^T:

1. Compute the high-dimensional transformed vectors φ(a) and φ(b).
2. Calculate their inner product φ(a)^T φ(b).
3. Verify that the polynomial kernel K(a, b) = (a^T b)^2 yields identical results.

Solution:

1. High-dimensional mapping: φ(a) = [2^2, sqrt(2)*2*3, 3^2]^T = [4, 6*sqrt(2), 9]^T φ(b) = [1^2, sqrt(2)*1*4, 4^2]^T = [1, 4*sqrt(2), 16]^T
2. Inner product: φ(a)^T φ(b) = (4 * 1) + (6*sqrt(2)* 4*sqrt(2)) + (9 * 16) φ(a)^T φ(b) = 4 + (24 * 2) + 144 = 4 + 48 + 144 = 196
3. Kernel Trick Verification: a^T b = (2 * 1) + (3 * 4) = 2 + 12 = 14 K(a, b) = (a^T b)^2 = 14^2 = 196 Both methods yield exactly 196.

Question 1.5 (Dual SVM Constraints and Coordinate Check) The dual optimization problem of a hard-margin SVM yields Lagrange multipliers α = [0.5, 0, 1.5, 0.2]^T corresponding to four training points with labels y = [+1, -1, -1, +1]^T.

1. Check if the fundamental linear equality constraint Σ (α_n * y_n) = 0 is satisfied.
2. Identify which data points are support vectors and which are non-support vectors based on the numerical α values.

Solution:

1. Equality constraint check: Σ (α_n * y_n) = (α1*y1) + (α2*y2) + (α3*y3) + (α4*y4) Σ (α_n * y_n) = (0.5 * 1) + (0 * -1) + (1.5 * -1) + (0.2 * 1) Σ (α_n * y_n) = 0.5 + 0 - 1.5 + 0.2 = -0.8 Since -0.8 ≠ 0, this configuration violates the dual validity condition.
2. Support Vector Identification: Any point with α_n > 0 acts as a support vector.
    - x1 (α1 = 0.5 > 0) -> Support Vector
    - x2 (α2 = 0) -> Non-Support Vector (lies outside the margin)
    - x3 (α3 = 1.5 > 0) -> Support Vector

- x4 (α4 = 0.2 > 0) -> Support Vector

========== MODULE 2: NEAREST NEIGHBORS AND DECISION TREES - 5 NUMERICAL QUESTIONS

Question 2.1 (k-NN Classification with Mixed Euclidean Distance) You have a reference 2D training dataset with 4 samples:

- P1 = [1, 1], Class A
- P2 = [1, 3], Class B
- P3 = [2, 4], Class B
- P4 = [3, 1], Class A An unknown query instance arrives at X_test = [2, 2]. Compute the Euclidean distances to all training points and determine the classification label using 3-NN (k=3).

Solution: Euclidean distance d = sqrt((x1 - x_test1)^2 + (x2 - x_test2)^2)

- d(P1, X_test) = sqrt((1-2)^2 + (1-2)^2) = sqrt(1 + 1) = sqrt(2) ≈ 1.414
- d(P2, X_test) = sqrt((1-2)^2 + (3-2)^2) = sqrt(1 + 1) = sqrt(2) ≈ 1.414
- d(P3, X_test) = sqrt((2-2)^2 + (4-2)^2) = sqrt(0 + 4) = sqrt(4) = 2.000
- d(P4, X_test) = sqrt((3-2)^2 + (1-2)^2) = sqrt(1 + 1) = sqrt(2) ≈ 1.414

The three nearest neighbors are P1, P2, and P4 (all sharing distance 1.414). Class distribution among neighbors:

- P1 -> Class A
- P2 -> Class B
- P4 -> Class A Class A has 2 votes, Class B has 1 vote. By majority vote, X_test is assigned to Class A.

Question 2.2 (k-NN Distance Metrics: Manhattan vs Minkowski) Calculate the exact distance metrics between vector u = [1, 5, -2] and vector v = [4, 1, 2] under:

1. Manhattan Distance (L1 norm).
2. Minkowski Distance of order p=3.

Solution:

1. Manhattan Distance (L1): D_L1 = |1 - 4| + |5 - 1| + |-2 - 2| = |-3| + |4| + |-4| = 3 + 4 + 4 = 11
2. Minkowski Distance (p=3): D_L3 = (|1 - 4|^3 + |5 - 1|^3 + |-2 - 2|^3)^(1/3) D_L3 = (|-3|^3 + |4|^3 + |-4|^3)^(1/3) D_L3 = (27 + 64 + 64)^(1/3) = (155)^(1/3) ≈ 5.371

Question 2.3 (Decision Tree Parent Node Entropy Calculation) A dataset at a root node contains a total of 80 instances. 50 instances belong to Class 'Yes' and 30 instances belong to Class 'No'. Calculate the exact Shannon Entropy (base 2) of this parent node. (Use log2(0.625) ≈ -0.678 and log2(0.375) ≈ -1.415).

Solution: Probability metrics: p_yes = 50/80 = 0.625; p_no = 30/80 = 0.375. Entropy formula: H = - [p_yes * log2(p_yes) + p_no * log2(p_no)] H = - [0.625 * (-0.678) + 0.375 * (-1.415)] H = - [-0.42375 - 0.530625] = - [-0.954375] = 0.9544 bits

---

Question 2.4 (Information Gain across a Threshold Split) The root node described in Question 2.3 (H_parent = 0.9544) is split by a continuous feature threshold into two branches:

- Left Branch: 40 instances total (35 'Yes', 5 'No')
- Right Branch: 40 instances total (15 'Yes', 25 'No') Compute the weighted entropy of the split and the resulting Information Gain. (Given: H_left ≈ 0.5436, H_right ≈ 0.9544)

Solution: Total parent samples N = 80. Left branch N_L = 40, Right branch N_R = 40. Weighted Entropy of Split (H_split): H_split = (N_L / N) * H_left + (N_R / N) * H_right H_split = (40/80) * 0.5436 + (40/80) * 0.9544 H_split = (0.5 * 0.5436) + (0.5 * 0.9544) = 0.2718 + 0.4772 = 0.7490

Information Gain: IG = H_parent - H_split = 0.9544 - 0.7490 = 0.2054 bits

---

Question 2.5 (Gini Impurity and Gini Gain Optimization) A node has 20 samples: 10 of Class Red and 10 of Class Blue. It is partitioned into:

- Left Child: 8 Red, 2 Blue
- Right Child: 2 Red, 8 Blue Calculate the initial parent Gini Impurity and the final Gini Gain.

Solution: Parent Gini Impurity: P_red = 10/20 = 0.5, P_blue = 10/20 = 0.5 G_parent = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 0.5

Child Nodes Gini:

- Left Child (N=10): P_red = 8/10 = 0.8, P_blue = 2/10 = 0.2 G_left = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.32
- Right Child (N=10): P_red = 2/10 = 0.2, P_blue = 8/10 = 0.8 G_right = 1 - (0.2^2 + 0.8^2) = 1 - (0.04 + 0.64) = 1 - 0.68 = 0.32

Weighted Gini of Split: G_split = (10/20) * 0.32 + (10/20) * 0.32 = 0.16 + 0.16 = 0.32

Gini Gain: G_gain = G_parent - G_split = 0.50 - 0.32 = 0.18

==========

========== MODULE 3: ENSEMBLE METHODS (BAGGING AND BOOSTING) - 5 NUMERICAL QUESTIONS

Question 3.1 (AdaBoost Error and Alpha Weight Calculation) In an AdaBoost training round, a weak classifier is evaluated over a dataset of 5 samples. The current sample weights are w = [0.1, 0.3, 0.2, 0.2, 0.2]. The weak classifier makes incorrect predictions on sample 2 and sample 4.

1. Calculate the total weighted error rate ε.
2. Determine the exact classifier update alpha weight α.

Solution:

1. Weighted Error Rate ε: ε = Σ (w_i * Indicator(misclassified)) The misclassified indices are 2 and 4. ε = w2 + w4 = 0.3 + 0.2 = 0.5
2. Classifier Weight α Calculation: α = 0.5 * ln((1 - ε) / ε) α = 0.5 * ln((1 - 0.5) / 0.5) = 0.5 * ln(1) = 0 An error rate of exactly 0.5 gives the classifier an importance weight of 0.

Question 3.2 (AdaBoost Sample Weights Multiplicative Update) Suppose an AdaBoost iteration has a weighted error ε = 0.20, resulting in a classifier weight α = 0.693. A subset of training points has a prior weight distribution w_old = 0.10. Compute the new unnormalized sample weight w_new for:

1. A sample that was correctly classified.
2. A sample that was misclassified.

Solution: The multiplicative factor rules for unnormalized updates are:

- Correct sample factor: exp(-α) = exp(-0.693) ≈ 0.50
- Incorrect sample factor: exp(α) = exp(0.693) ≈ 2.00

Weight Calculations:

1. For Correctly Classified Sample: w_new = w_old * exp(-α) = 0.10 * 0.50 = 0.05
2. For Misclassified Sample: w_new = w_old * exp(α) = 0.10 * 2.00 = 0.20

Question 3.3 (Normalization of Exponentially Updated Weights) Following an update step, a vector of unnormalized weights for a 4-sample dataset is computed as w_unnorm = [0.04, 0.12, 0.04, 0.20]. Normalize these values to sum to 1.

Solution: Calculate the sum of all unnormalized weights: Sum = 0.04 + 0.12 + 0.04 + 0.20 = 0.40

Divide each component by the total sum:

- w1 = 0.04 / 0.40 = 0.10
- w2 = 0.12 / 0.40 = 0.30
- w3 = 0.04 / 0.40 = 0.10
- w4 = 0.20 / 0.40 = 0.50 The normalized weight vector is w_norm = [0.10, 0.30, 0.10, 0.50].

Question 3.4 (Ensemble Majority Vote Output Calculation) A Random Forest ensemble contains 5 trained decision trees. For a test input vector x, the individual trees predict class labels as follows: Tree1='Class 0', Tree2='Class 1', Tree3='Class 1', Tree4='Class 0', Tree5='Class 1'. Calculate the vote proportions and final ensemble prediction.

Solution: Count votes for each class label:

- Total Votes for Class 0 = 2 (Tree1, Tree4)
- Total Votes for Class 1 = 3 (Tree2, Tree3, Tree5)

Proportion calculations:

- P(Class 0) = 2/5 = 0.40 (40%)
- P(Class 1) = 3/5 = 0.60 (60%) By majority vote decision rule, the final ensemble prediction is Class 1.

Question 3.5 (Weighted Ensemble Sign-Based Inference) An AdaBoost ensemble consists of three weak classifiers with voting weights α1 = 0.8, α2 = 0.5, and α3 = 1.1. For an incoming test example, the base models output predictions h1(x) = +1, h2(x) = -1, and h3(x) = -1. Compute the exact final score and sign-based output.

Solution: The ensemble score equation is: Score = Σ (α_t * h_t(x)) Score = (α1 * h1) + (α2 * h2) + (α3 * h3) Score = (0.8 * 1) + (0.5 * -1) + (1.1 * -1) Score = 0.8 - 0.5 - 1.1 = -0.8

Final predicted label = sign(Score) = sign(-0.8) = -1.

========== MODULE 4: NEURAL NETWORKS AND BACKPROPAGATION - 5 NUMERICAL QUESTIONS

Question 4.1 (Scalar Backpropagation Step with Linear Graph) Consider a mathematical node processing two inputs x = 4, w = -3 through a function f(x, w) = w * x. During backpropagation, the incoming upstream derivative from the loss layer is dL/df = -2.0. Compute the local derivatives and downstream gradients dL/dw and dL/dx.

Solution: Local derivatives of the multiplication function:

- df/dx = w = -3
- df/dw = x = 4

Apply the chain rule to propagate the loss gradient down:

- dL/dx = dL/df * df/dx = (-2.0) * (-3) = 6.0
- dL/dw = dL/df * df/dw = (-2.0) * (4) = -8.0

Question 4.2 (Sigmoid Node Activation and Gradient Optimization) A neuron calculates a local activation output using a standard sigmoid function: σ(z) = 1 / (1 + exp(-z)).

1. Given a linear activation sum value z = 0, calculate the exact numeric forward pass output a = σ(z).
2. If the upstream derivative is dL/da = 4.0, calculate the downstream gradient dL/dz.

Solution:

1. Forward activation pass: a = 1 / (1 + exp(-0)) = 1 / (1 + 1) = 0.5
2. Sigmoid Local Gradient property: da/dz = a * (1 - a) da/dz = 0.5 * (1 - 0.5) = 0.25 Downstream loss gradient computation via chain rule: dL/dz = dL/da * da/dz = 4.0 * 0.25 = 1.0

Question 4.3 (ReLU Node Deflection Gradient Mechanics) A hidden layer computational unit applies a ReLU activation function: f(z) = max(0, z). Compute the downstream loss gradient dL/dz for two alternative tracking scenarios:

- Scenario A: z = 2.50 with upstream gradient dL/df = 3.5
- Scenario B: z = -1.25 with upstream gradient dL/df = 3.5

Solution: The derivative of ReLU is: df/dz = 1 if z > 0 else 0.

- Scenario A (z = 2.50 > 0): The local derivative df/dz = 1. dL/dz = dL/df * df/dz = 3.5 * 1 = 3.5
- Scenario B (z = -1.25 < 0): The local derivative df/dz = 0. dL/dz = dL/df * df/dz = 3.5 * 0 = 0

Question 4.4 (Weight Matrix Gradient using Matrix Calculus Identity) A layer output matrix is defined by the transformation Y = XW. The input activation matrix X and upstream loss matrix dL/dY are given as: X = [[2, 1], [0, 3]] dL/dY = [[1, 4], [2, 0]] Compute the exact matrix components of the weight parameter gradient dL/dW = X^T * dL/dY.

Solution: First, transpose the matrix X: X^T = [[2, 0], [1, 3]]

Perform matrix multiplication between X^T and dL/dY: Row 1, Col 1: (2 * 1) + (0 * 2) = 2 Row 1, Col 2: (2 * 4) + (0 * 0) = 8 Row 2, Col 1: (1 * 1) + (3 * 2) = 1 + 6 = 7 Row 2, Col 2: (1 * 4) + (3 * 0) = 4

The resulting gradient matrix is: dL/dW = [[2, 8], [7, 4]]

Question 4.5 (Input Layer Matrix Gradient Calculations) Using the same parameters and input matrices from Question 4.4, compute the input gradient matrix dL/dX using the matrix identity dL/dX = dL/dY * W^T. Assume the layer weight matrix W is defined as: W = [[1, 2], [3, 4]]

Solution: First, transpose the weight matrix W: W^T = [[1, 3], [2, 4]]

Multiply the upstream gradient matrix dL/dY by W^T: dL/dX = [[1, 4], * [[1, 3], [2, 0]] [2, 4]]

Row 1, Col 1: (1 * 1) + (4 * 2) = 1 + 8 = 9 Row 1, Col 2: (1 * 3) + (4 * 4) = 3 + 16 = 19 Row 2, Col 1: (2 * 1) + (0 * 2) = 2 Row 2, Col 2: (2 * 3) + (0 * 4) = 6

The resulting input gradient matrix is: dL/dX = [[9, 19], [2, 6]]

========== MODULE 5: CONVOLUTIONAL NEURAL NETWORKS (CNNS) - 5 NUMERICAL QUESTIONS

Question 5.1 (CNN Output Shape Dimension Math) An input feature map matrix tensor has dimensions [Width = 16, Height = 16, Channels = 3]. It is processed by a convolutional layer with the following properties:

- Filter Count (K) = 16
- Spatial Filter Footprint (F) = 5
- Operational Stride (S) = 1
- Symmetric Padding (P) = 2 Calculate the exact spatial matrix dimensions [W_out, H_out, C_out] of the output feature tensor.

Solution: Apply the spatial dimension formula: Out = floor((In - F + 2P) / S) + 1 W_out = floor((16 - 5 + 2(2)) / 1) + 1 = floor(15 / 1) + 1 = 16 Due to spatial symmetry, H_out = 16. The channel depth matches the number of convolutional filters: C_out = K = 16. The output tensor dimensions are [16, 16, 16].

Question 5.2 (Strided Convolution Output Dimension Reduction) Recalculate the spatial dimensions [W_out, H_out, C_out] for the input tensor in Question 5.1 if the stride is changed to S = 2, and padding is removed (P = 0).

Solution: W_out = floor((In - F + 2P) / S) + 1 W_out = floor((16 - 5 + 2(0)) / 2) + 1 = floor(11 / 2) + 1 = 5 + 1 = 6 H_out = 6 by symmetry. C_out remains equal to the number of filters K = 16. The output tensor dimensions are [6, 6, 16].

Question 5.3 (CNN Parameter Count Calculation) Calculate the total number of learnable parameters (weights and biases) for the convolutional layer described in Question 5.1 (Input Channels = 3, Filter Count K = 16, Filter Size F = 5). Assume each filter has one bias term.

Solution: Parameters per single filter: Weights = Filter_Width * Filter_Height * Input_Channels = 5 * 5 * 3 = 75 parameters. Each filter has exactly 1 bias parameter, so: Total parameters per filter = 75 + 1 = 76 parameters.

Since there are K = 16 filters in the layer: Total Layer Parameters = 16 * 76 = 1216 parameters.

Question 5.4 (Max Pooling Layer Operation and Shape Transformation) An output activation tensor map of size [8 x 8 x 4] is passed into a Max Pooling layer with a filter size of F = 2 and a stride of S = 2.

1. Determine the spatial output dimensions.
2. Given a 2x2 slice window of an activation matrix: [[1.2, 0.5], [2.1, 1.8]] Compute the exact numeric value produced by the pooling operation.

Solution:

1. Spatial output dimension for pooling: Out = ((In - F) / S) + 1 = ((8 - 2) / 2) + 1 = 3 + 1 = 4 The output shape is [4 x 4 x 4].

2. Max Pooling extracts the maximum value within the window: Value = max(1.2, 0.5, 2.1, 1.8) = 2.1

Question 5.5 (Receptive Field Growth Calculation) A deep CNN architecture stacks three consecutive convolutional layers directly on top of each other. All layers use a filter size of F = 3 and a stride of S = 1. Calculate the receptive field size of a single neuron in Layer 3 relative to the original input space.

Solution: Receptive field growth formula: RF_now = RF_prev + (F - 1) * Stride_Product

- Layer 1 (Base): RF_1 = 3

- Layer 2: RF_2 = 3 + (3 - 1) * 1 = 5
- Layer 3: RF_3 = 5 + (3 - 1) * 1 = 7 A single neuron in Layer 3 has a receptive field of 7x7 pixels relative to the input image.

========== MODULE 6: RECURRENT NEURAL NETWORKS (RNN) - 5 NUMERICAL QUESTIONS

Question 6.1 (Vanilla RNN Hidden State Update Step) A Vanilla Recurrent Neural Network hidden neuron updates its state using the formula: h_t = tanh(W_hh * h_prev + W_xh * x_t + b). Given the following parameters:

- Recurrent scalar weight W_hh = 0.4
- Input scalar weight W_xh = 1.5
- Bias term b = -0.2 Calculate the net input sum z_t and the final hidden state h_t at time step t if the previous hidden state was h_prev = 0.50 and the current input is x_t = 1.00. (Use the approximation tanh(1.5) ≈ 0.905).

Solution: Calculate the net input sum z_t before activation: z_t = (W_hh * h_prev) + (W_xh * x_t) + b z_t = (0.4 * 0.50) + (1.5 * 1.00) + (-0.2) = 0.20 + 1.50 - 0.2 = 1.50

Apply the activation function: h_t = tanh(1.50) ≈ 0.905

Question 6.2 (RNN Unrolled Sequence Forward Pass Computation) Using the same network parameters from Question 6.1 (W_hh = 0.4, W_xh = 1.5, b = -0.2), calculate the next hidden state h_{t+1} if the next input token value is x_{t+1} = -0.50. (Use tanh(-0.588) ≈ -0.528).

Solution: The previous hidden state is now h_t = 0.905. Calculate the new net input sum z_{t+1}: z_{t+1} = (W_hh * h_t) + (W_xh * x_{t+1}) + b z_{t+1} = (0.4 * 0.905) + (1.5 * -0.50) - 0.2 = 0.362 - 0.75 - 0.2 = -0.588

Apply the activation function: h_{t+1} = tanh(-0.588) ≈ -0.528

Question 6.3 (RNN Exploding Gradient Sequence Verification) A deep unrolled recurrent sequence has an identical weight derivative chain across time-steps. The total gradient expression contains the term (W_hh)^T. If W_hh = 2.50 and the sequence length is T = 4, compute the product multiplier to illustrate why gradients explode.

Solution: The multiplicative growth factor across 4 steps is: Factor = (W_hh)^4 = (2.50)^4 = 39.0625 Because the parameter value is greater than 1, the gradient is amplified significantly.

Question 6.4 (RNN Vanishing Gradient Sequence Verification) Recalculate the gradient growth factor from Question 6.3 if the recurrent weight is reduced to W_hh = 0.25 across a sequence length of T = 4 steps.

Solution: The multiplicative growth factor across 4 steps is: Factor = (W_hh)^4 = (0.25)^4 = 0.00390625 Because the parameter value is less than 1, the gradient decays significantly toward 0.

Question 6.5 (LSTM Gate Output Computation Mechanics) An LSTM cell uses a sigmoid function to determine its forget gate activation: f_t = σ(W_f * [h_prev, x_t] + b_f). Given the linear combined sum input to the forget gate is exactly 0.00, calculate:

1. The numeric gate activation level f_t.
2. The new cell state C_t if the previous cell state was C_prev = 6.0 and the input candidate update is completely blocked.

Solution:

1. Forget gate activation: f_t = σ(0.00) = 1 / (1 + exp(-0)) = 0.50 (The gate is 50% open).
2. Cell state update: Assuming the input gate is closed (candidate = 0): C_t = f_t * C_prev + 0 = 0.50 * 6.0 = 3.0

==========

========== MODULE 7: CLUSTERING AND GAUSSIAN MIXTURE MODELS (GMM) - 5 NUMERICAL QUESTIONS

Question 7.1 (k-means Cluster Centroid Distance Assignment) You are running a k-means clustering algorithm with K=2. There are two initial cluster centroids:

- μ1 = [1.0, 1.0]
- μ2 = [4.0, 4.0] A data point is located at x = [2.0, 3.0]. Calculate the squared Euclidean distance from x to both centroids and assign the point to the closest cluster.

Solution: Squared Euclidean distance formula: d^2 = (x1 - μ_1)^2 + (x2 - μ_2)^2

- Distance to Centroid 1: d^2(x, μ1) = (2.0 - 1.0)^2 + (3.0 - 1.0)^2 = 1^2 + 2^2 = 5.0
- Distance to Centroid 2: d^2(x, μ2) = (2.0 - 4.0)^2 + (3.0 - 4.0)^2 = (-2)^2 + (-1)^2 = 5.0

Since both distances are exactly equal (5.0 = 5.0), we default to the lower index cluster. The point is assigned to Cluster 1.

Question 7.2 (k-means Centroid Coordinate Update Calculation) A cluster contains three data points after the assignment step: A = [1, 2], B = [2, 3], and C = [3, 7]. Compute the updated coordinates for this cluster's centroid vector μ_new.

Solution: The updated centroid is the mean of the coordinates of all assigned points: μ_new = [(x_A + x_B + x_C)/3 , (y_A + y_B + y_C)/3] μ_new = [(1 + 2 + 3)/3 , (2 + 3 + 7)/3] = [2.0, 4.0]

Question 7.3 (k-means++ Probability Initialization Math) In a k-means++ initialization step over a 3-point dataset, the squared distances from each point to the closest existing center are:

- d^2(x1) = 2.0
- d^2(x2) = 8.0
- d^2(x3) = 0.0 Calculate the exact selection probability distribution [p1, p2, p3] for choosing the next cluster center.

Solution: Total sum of squared distances: Sum = 2.0 + 8.0 + 0.0 = 10.0 Selection probability formula: p_i = d^2(x_i) / Sum

- p1 = 2.0 / 10.0 = 0.20 (20%)
- p2 = 8.0 / 10.0 = 0.80 (80%)
- p3 = 0.0 / 10.0 = 0.00 (0%) The probability distribution is [0.20, 0.80, 0.00].

Question 7.4 (GMM Soft Responsibility Component Math) In a 2-component Gaussian Mixture Model, the weighted probability densities for a data point x are calculated as:

- Component 1: π1 * N(x | μ1, Σ1) = 0.60
- Component 2: π2 * N(x | μ2, Σ2) = 0.20 Calculate the soft responsibility metrics γ_1 and γ_2 that Component 1 and Component 2 share for this data point.

Solution: Total density sum = 0.60 + 0.20 = 0.80 Responsibility formula: γ_k = (π_k * N_k) / Sum

- γ_1 = 0.60 / 0.80 = 0.75 (75% responsibility assigned to Component 1)
- γ_2 = 0.20 / 0.80 = 0.25 (25% responsibility assigned to Component 2)

Question 7.5 (GMM Parameter Maximum Likelihood Update Metric) The soft responsibility values assigned to Component 1 across a 3-point dataset are γ = [0.5, 1.0, 0.5]. The data point values are x1 = 2.0, x2 = 4.0, and x3 = 6.0. Compute the updated mean μ_1 for Component 1.

Solution: μ_1 = Σ (γ_i * x_i) / Σ (γ_i) Denominator (sum of responsibilities): Sum_γ = 0.5 + 1.0 + 0.5 = 2.0 Numerator (weighted sum of data points): Num = (0.5 * 2.0) + (1.0 * 4.0) + (0.5 * 6.0) = 8.0 μ_1 = 8.0 / 2.0 = 4.0
